

Network Security Automata Simulator

Arriba, Butaya, Gamale, Rollan, Sorongon
BSCS – 3B

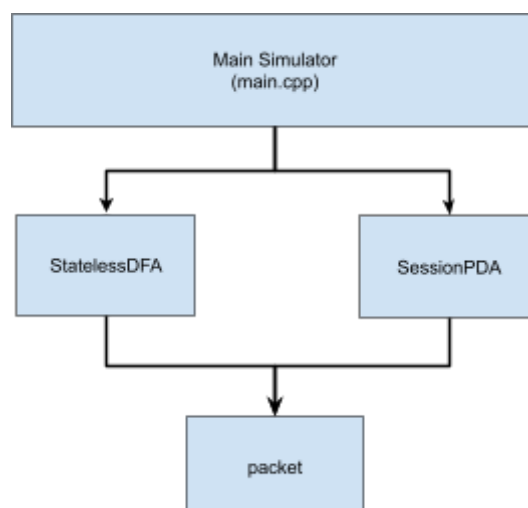
1. Overview

This simulator demonstrates network security using formal language theory and automata-based pattern matching. At its core, the simulator implements two different types of automaton, each designed to address specific categories of security threats:

- *Deterministic Finite Automaton (DFA)* - Implemented via DFA to scan packet payloads for static malicious signatures (e.g., "root", NOP sleds).
- *Pushdown Automaton (PDA)* - Type-2 Context-Free Grammar validator Implemented via PDA to validate the recursive or state-dependent nature of network protocols (e.g., verifying a SYN is followed by an ACK).

The system ensures that every network packet will go through analysis by passing through both automata layers simultaneously, creating a robust defense mechanism where traffic is blocked quickly if it violates either the DFA and PDA. This approach ensures that even if one layer fails to detect an attack, the other layer may still catch it.

2. Simulation Diagram



Flow:

1. *Packet creation*: Test scenarios generate packets with specific characteristics
2. *DFA analysis*: Pattern matching against known malicious signatures
3. *PDA analysis*: Protocol state validation using stack-based context tracking
4. *Decision*: Block if automaton rejects, allow only if both automaton accept

3. Simulation Components

3.1 Packets (packet.h file)

The packet is the core data container representing a single unit of network communication in the simulation, containing important information including addressing information, control flags, and the actual data payload being transmitted across the network.

```
struct Packet {  
    string sourceIP;        // Source IP address identifying packet origin  
    string destIP;         // Destination IP address identifying target system  
    int flags;              // TCP-style control flags using bitwise encoding  
    string payload;         // Actual data content being transmitted  
};
```

TCP Flags (Bitwise Encoding)

The control flags employ a bitwise encoding scheme enabling multiple flags to be set at once within a single integer, efficiently representing multiple boolean states with minimal memory, similar to TCP headers.

```
enum Flags {  
    SYN = 1 << 0, // 0001 - Synchronize (connection start)  
    ACK = 1 << 1, // 0010 - Acknowledge  
    FIN = 1 << 2, // 0100 - Finish (connection end)  
    RST = 1 << 3, // 1000 - Reset  
};
```

3.2 Stateless DFA (Pattern Matcher)

The StatelessDFA class implements a pattern matcher designed to reject packets containing specific malicious byte sequences. It scans the payload byte-by-byte without memory of previous packets.

Formal Definition:

Q (States): {q0,q1,q2,q3,q4,q5,q6,q7}

q0: Initial safe state.

q1,q2,q3: Intermediate states detecting the string "root".

q4: Trap state detecting "root" (Malicious Text).

q5,q6: Intermediate states detecting NOP sleds (\x90).

q7: Trap state detecting NOP sleds (Binary Exploit).

Σ (Alphabet): The set of all ASCII characters and binary byte values (e.g., 0x90).

q0 (Start State): State 0.

F (Accept/Reject States): {4,7}

In this context, reaching a "Final" state implies detecting a threat, resulting in packet rejection.

Transition Function (δ):

Current State	Input (c)	Next State	Semantics
q0	'r'	q1	Start of "root"
q0	0x90	q5	Start of Buffer Overflow
q0	Other	q0	Reset
q1 ('r')	'o'	q2	"ro..."
q2 ('ro')	'o'	q3	"roo..."
q3 ('roo')	't'	4	"root" (Detected)
q5 (\x90)	0x90	6	Double NOP
q6 (\x90\x90)	0x90	q7	NOP Sled

			(Detected)
q4, q7	Any	q4, q7	Trap (Permanently Blocked)

3.3 Session PDA (Protocol Validator)

The SessionPDA class implements a stateful validator that ensures packets arrive in a valid TCP-like handshake order. It uses a stack to enforce dependencies (e.g., Data cannot be sent before a Session is active).

Formal Definition:

Q (States): {CLOSED, SYN_SENT, ESTABLISHED}

Σ (Input Alphabet): {SYN, ACK, DATA} (derived from packet flags and payload presence).

Γ (Stack Alphabet): {WAIT_ACK, ACTIVE}

q0 (Start State): CLOSED

Z0 (Initial Stack Symbol): Z0 (Bottom Marker)

F (Accepting Criteria): Processing packet without rejection error.

Transition Logic (δ):

Handshake Initiation (SYN):

$\delta(\text{CLOSED}, \text{SYN}, \epsilon) \rightarrow (\text{SYN_SENT}, \text{push}(\text{WAIT_ACK}))$

Constraint: Top of stack must be Z0.

Handshake Completion (ACK):

$\delta(\text{SYN_SENT}, \text{ACK}, \text{WAIT_ACK}) \rightarrow (\text{ESTABLISHED}, \text{pop}(\text{WAIT_ACK}); \text{push}(\text{ACTIVE}))$

Constraint: If ACK arrives and top is Z0, reject (ACK Scan). If top is WAIT_ACK, proceed.

Data Transmission:

$\delta(\text{ESTABLISHED}, \text{DATA}, \text{ACTIVE}) \rightarrow (\text{ESTABLISHED}, \text{no_op})$

Constraint: Top of stack must be ACTIVE.

4. State Diagrams

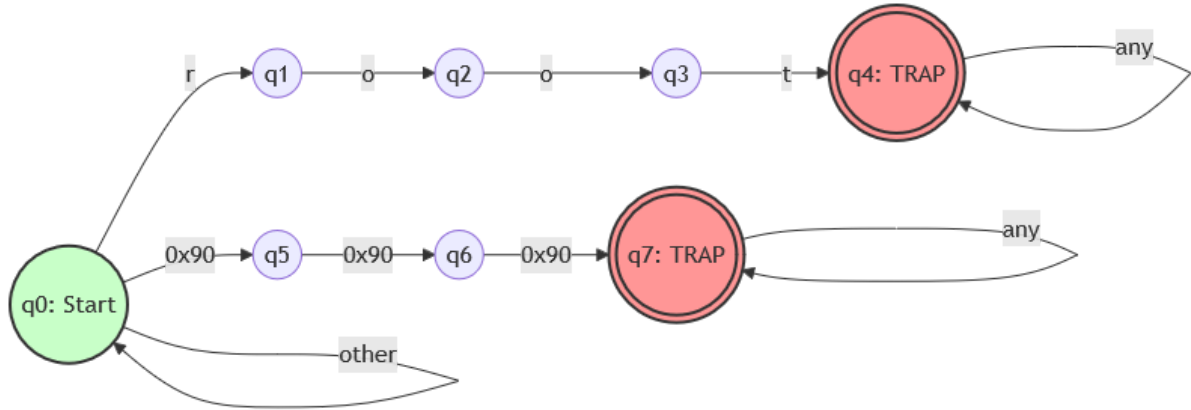


Figure 4.1 DFA Diagram (Payload Scanning)

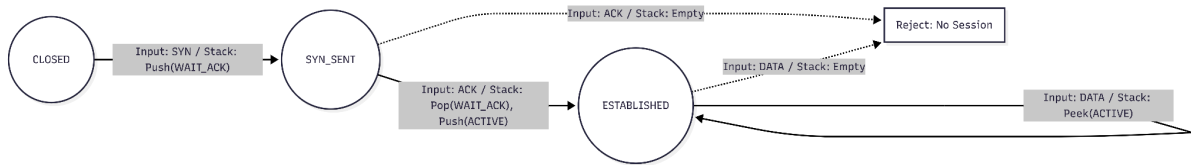


Figure 4.2 PDA Diagram (Session Logic)

5. Threat Modeling and Attack Signatures

This section defines the specific security threats targeted by the proposed automata. The system employs a dual-layer detection strategy: signature-based detection for malicious payloads via the **Stateless DFA**, and anomaly-based detection for protocol state violations via the **Session PDA**.

Stateless Detection: Malicious Payload Signatures

The **Stateless Deterministic Finite Automaton (DFA)** operates on the principle of pattern matching, scanning individual packet payloads for byte sequences indicative of known attack vectors. It targets two primary categories of exploitation: administrative privilege escalation and binary code execution.

Text-Based Privilege Escalation Heuristics ("root")

- **Signature Definition:** The character sequence root appearing within a plaintext payload.
- **Attack Vector:** In Unix-like operating systems, the root account possesses unrestricted superuser privileges. The presence of this string in a network stream acts as a heuristic indicator for:
- **Brute-Force Authentication:** Automated scripts attempting to compromise SSH, Telnet, or FTP services by systematically guessing credentials for the default administrative account.
- **Privilege Escalation Attempts:** Malicious command execution (e.g., su root or sudo) aimed at elevating user privileges post-compromise.
- **Cleartext Credential Exposure:** inadvertent transmission of administrative credentials over unencrypted channels, violating data confidentiality protocols.

Binary Exploitation: The NOP Sled

- **Signature Definition:** A contiguous sequence of the hexadecimal byte 0x90 (e.g., 0x90 0x90 0x90...).
- **Attack Vector:** In the x86 instruction set architecture, 0x90 corresponds to the **NOP** (No Operation) instruction. This sequence is a hallmark of **Buffer Overflow** attacks, specifically the "NOP Sled" technique.
- **Mechanism of Action:** Attackers prefix malicious shellcode with a long run of NOP instructions. By corrupting the instruction pointer (EIP) to land anywhere within this "sled," the processor executes the NOPs sequentially until it reaches and executes the payload.
- **Detection Logic:** Legitimate ASCII text or compressed data rarely generates long, contiguous sequences of 0x90 bytes. Consequently, the detection of such a sequence serves as a high-fidelity indicator of an active stack-based buffer overflow attempt.

Stateful Detection: Protocol Anomaly Signatures

The **Session Pushdown Automaton (PDA)** implements stateful inspection utilizing a stack memory structure to enforce the Transmission Control Protocol (TCP) finite state machine. It identifies Protocol State Anomalies, events that violate the strict ordering of the TCP Three-Way Handshake.

Unsolicited Response Detection (Flow Violation)

- **Protocol Specification:** The TCP connection establishment phase requires that an Acknowledgment (ACK) or Synchronize-Acknowledgment (SYN-ACK) packet be preceded by an initial Synchronize (SYN) request from the initiating client.
- **Anomaly Condition:** The firewall detects an inbound ACK or SYN-ACK packet while the internal state stack is empty (Null State), indicating the absence of a prerequisite connection request.
- **Security Implications:**
 1. **TCP ACK Scanning:** Adversaries employ "ACK scans" to map network topology. By sending random ACK packets to ports, they

analyze the resulting RST (Reset) packets to determine firewall rules and host availability.

2. **Firewall Evasion (Hole Punching):** Attackers transmit spoofed ACK packets to deceive stateless packet filters into believing a connection is already established, thereby bypassing ingress filtering rules.
3. **Backscatter/Reflected Attacks:** Unsolicited responses may indicate the network is being targeted by backscatter from a spoofed Denial of Service (DoS) attack occurring elsewhere.

TCP State Desynchronization (Sequence Invalidity)

- **Protocol Specification:** The flags present in a TCP segment must strictly correspond to the current connection state maintained at the top of the stack. For instance, a SYN-ACK is only valid if the session is in the SYN_SENT state.
- **Anomaly Condition:** The system receives a transition signal (e.g., FIN or Data ACK) that is invalid for the current stack context (e.g., while waiting for a handshake completion).
- **Security Implications:**
 1. **TCP Session Hijacking:** A sequence mismatch suggests that an external entity is attempting to inject packets into an existing session. If the attacker fails to predict the correct sequence number or state, the packet will violate the PDA's expected transition.
 2. **Blind Reset Attacks:** An attacker may attempt to terminate active connections by spoofing RST or FIN packets. The PDA detects these as anomalies if the connection state does not logically permit a termination sequence at that specific interval.
 3. **Implementation Fingerprinting:** Deviations in state transitions can sometimes imply non-standard TCP stack implementations used by specific malware variants or scanning tools.

6. Simulation Flow

The simulation flow illustrates the executed test scenarios demonstrating the security capabilities of the dual-automata system. Understanding this flow is crucial for comprehending the interaction of components and the detection of various attack patterns through the coordinated operation of the security layers.

Test Scenario 1: Malicious Payload Detection

This scenario illustrates the DFA's capability to identify malicious keywords in packet payloads, highlighting Type-3 regular language pattern matching. It simulates an attacker who successfully connects using the TCP handshake but tries to compromise the system by sending a payload with suspicious keywords.

```
vector<Packet> createScenario_MaliciousContent() {
    return {
        {"1.2.3.4", "10.0.0.1", SYN, ""},
        {"1.2.3.4", "10.0.0.1", ACK, ""},
        {"1.2.3.4", "10.0.0.1", 0, "I want root access"}
    };
}
```

Execution Trace:

```
vector<Packet> createScenario_MaliciousContent() {
Packet 1: SYN
├─ DFA: No payload → PASS
└─ PDA: Stack[] → Stack[WAIT_ACK] → State: SYN_SENT

Packet 2: ACK
├─ DFA: No payload → PASS
└─ PDA: Stack[WAIT_ACK] → Pop → Push[ACTIVE] → State: ESTABLISHED

Packet 3: DATA "I want root access"
├─ DFA: Scans byte-by-byte:
│   I → w → a → n → t → [space] → r → o → o → t
│   State transitions: 0→0→0→...→0→1→2→3→4 [TRAP!]
│   BLOCKED: "root" detected
└─ PDA: Stack[ACTIVE] → Session valid → PASS
}
```

Result: DFA blocks packet before PDA analysis matters.

Test Scenario 2: Protocol Violation Detection (ACK Scan Attack)

This scenario illustrates the PDA's ability to identify protocol-level attacks, specifically highlighting an ACK scan attack. This type of attack, characterized by the sending of ACK packets without proper connection establishment, evades detection by content-based filters as it does not contain any harmful content, but rather violates protocol sequencing rules.

```
vector<Packet> createScenario_OutOfOrder() {
    return {
        {"1.2.3.4", "10.0.0.1", 0, "Hello Server"}
    };
}
```

Execution Trace:

```
vector<Packet> createScenario_OutOfOrder() {
    Packet 1: DATA (No handshake)
    DFA: Payload = "Hello Server"
        Scans: H-e-l-l-o...
        No "root" or NOP sled detected
        PASS

    PDA: Check stack for SESSION ACTIVE
        Stack contains: [Z0] (Bottom Marker)
        Condition Failed: Top is not ACTIVE
        BLOCKED: Data without handshake

}
```


Result: DFA passes, PDA blocks. Demonstrates that valid text is rejected if protocol is violated.

Test Scenario 3: Binary Exploit Detection (NOP Sled)

The third and most sophisticated scenario demonstrates the DFA's capability to detect binary-level exploits, specifically buffer overflow attempts characterized by NOP sleds. This scenario showcases how the same pattern matching principles used for text-based signatures can be extended to identify byte-level exploit patterns that appear in actual malware and exploit code.

```
vector<Packet> createScenario_BufferOverflow() {
    string exploitPayload = "\x90\x90\x90\x90\x90"; // 5 NOPs
    exploitPayload += "\xCC\xCC"; // INT 3 shellcode

    return {
        {"6.6.6.6", "10.0.0.1", SYN, ""},
        {"6.6.6.6", "10.0.0.1", ACK, ""},
        {"6.6.6.6", "10.0.0.1", 0, exploitPayload}
    };
}
```

Execution Trace:

```
Packet 1-2: SYN/ACK handshake
└─ Both DFA and PDA accept (establishes valid session)

Packet 3: Binary payload
└─ DFA: Scans hex bytes:
    │   \x90 → State 5
    │   \x90 → State 6
    │   \x90 → State 7 [TRAP!]
    │   BLOCKED: NOP sled detected (shellcode signature)
└─ PDA: Stack[ACTIVE] → Would pass protocol check
```

Result: Legitimate session, but DFA catches exploit attempt via binary signature.

Test Scenario 4: False Positive Check (Safe Context)

Verify that the DFA correctly distinguishes between malicious signatures (e.g., "root") and safe words that share prefixes (e.g., "route").

```
vector<Packet> createScenario_FalsePositive() {
    vector<Packet> traffic;
    traffic.push_back({"192.168.1.5", "10.0.0.1", SYN, ""});
    traffic.push_back({"192.168.1.5", "10.0.0.1", ACK, ""});

    traffic.push_back({"192.168.1.5", "10.0.0.1", 0, "Checking route
path..."});
    return traffic;
}
```

Execution Trace:

```
vector<Packet> createScenario_FalsePositive() {
    Packet 1-2: SYN/ACK handshake
    Both DFA and PDA accept (establishes valid session)

    Packet 3: DATA "Checking route path..."
    DFA: Scans byte-by-byte:
        ... [space]
        r -> State q1 (Potential threat start)
        o -> State q2 ("ro" matches prefix)
        u -> State q0 (RESET! 'u' does not match 'o' for "root")
        t -> State q0 (Safe)
        e -> State q0 (Safe)
        PASS

    PDA: Stack [ACTIVE]
        Session valid
        PASS

}
```

Result: Packet Allowed. The DFA correctly distinguished "route" from "root" by resetting state on the character 'u'.

Test Scenario 5: Split Attack (Fragmentation)

Demonstrate the inherent weakness of a Stateless DFA when a malicious payload is split across multiple packets.

```
vector<Packet> createScenario_SplitAttack() {
    vector<Packet> traffic;
    traffic.push_back({"192.168.1.5", "10.0.0.1", SYN, ""});
    traffic.push_back({"192.168.1.5", "10.0.0.1", ACK, ""});

    traffic.push_back({"192.168.1.5", "10.0.0.1", 0, "User: ro"});
    traffic.push_back({"192.168.1.5", "10.0.0.1", 0, "ot access"});
    return traffic;
}
```

Execution Trace:

```
vector<Packet> createScenario_SplitAttack() {
    Packet 1-2: SYN/ACK handshake
    Both DFA and PDA accept (establishes valid session)

    Packet 3: DATA "User: ro"
    DFA: Scans byte-by-byte:
        ... [space]
        r -> State q1
        o -> State q2 ("ro")
        End of Packet -> DFA State Resets to q0 (Stateless Property)
        PASS

    Packet 4: DATA "ot access"
    DFA: Scans byte-by-byte:
        o -> State q0 (Start state expects 'r', got 'o')
        t -> State q0
```

```
PASS (False Negative)

PDA: Stack [ACTIVE]
     Session valid
     PASS

}

}
```

Result: FALSE NEGATIVE. The firewall failed to detect "root" because the signature was split across two packets. This demonstrates the critical limitation of a Stateless DFA.

Test Scenario 6: XSS Attack Detection (Cross-Site Scripting)

This scenario is to demonstrate that the DFA scans payloads regardless of the application layer format (HTML/JS).

```
vector<Packet> createScenario_XSS() {
    vector<Packet> traffic;
    traffic.push_back({"10.10.10.10", "10.0.0.1", SYN, ""});
    traffic.push_back({"10.10.10.10", "10.0.0.1", ACK, ""});

    traffic.push_back({"10.10.10.10", "10.0.0.1", 0, "<script>var
user='root';</script>"});
    return traffic;
}
```

Execution Trace:

```
vector<Packet> createScenario_XSS() {
    Packet 1-2: SYN/ACK handshake
    Both DFA and PDA accept (establishes valid session)

    Packet 3: DATA "<script>var user='root';</script>"
    DFA: Scans byte-by-byte:
        < s c r i p t > ... (Safe characters, DFA loops on q0)
        ...
        r -> State q1
        o -> State q2
        o -> State q3
        t -> State q4 (TRAP!)
        BLOCKED: "root" detected embedded in script

    PDA: Stack [ACTIVE]
         Session valid
         PASS

}
```

Result: DFA blocks the packet. This demonstrates that the pattern matcher is protocol-agnostic and will catch signatures even when hidden inside HTML or JavaScript code.

